

Árbol de Decisión aplicado a la Enfermedad de Carrión

Explicación del código, paso a paso (material de exposición)

Inteligencia Artificial · UNTELS · Caso: enfermedad de Carrión (MINSA) · Comparativa Red Neuronal vs. Árbol de Decisión

Este documento explica, **bloque por bloque**, el código del modelo de **Árbol de Decisión** que clasifica la fase de la enfermedad de Carrión en **AGUDA** (0) o **ERUPTIVA** (1). Un árbol de decisión hace preguntas sucesivas sobre los datos (por ejemplo «¿edad < 15?») y, según las respuestas, desciende por sus ramas hasta llegar a una hoja con la decisión final.

1. Librerías y carga de datos

```
import pandas as pd, numpy as np
from sklearn import tree
training = pd.read_csv("comparador-ml/data/carrion-train.csv")
```

Usamos **scikit-learn** (módulo **tree**), la librería estándar de Machine Learning clásico en Python. Cargamos el mismo 80% de entrenamiento que usa la red neuronal: ambos modelos comparten la **misma fuente de datos** (requisito del examen para que la comparación sea justa).

2. Preprocesamiento (igual que la red, pero SIN normalizar)

```
df["edad_anios"] = df.apply(lambda r: edad_anios(r["edad"], r["tipo_edad"]), axis=1)
df["sexo_cod"] = df["sexo"].apply(lambda s: 0 if str(s).upper()=="M" else 1)
df["depto_cod"] = df["departamento"].apply(lambda d: depto_idx[d])
df["target"] = df["enfermedad"].map({"...AGUDA":0, "...ERUPTIVA":1})
columnas = ["edad_anios", "sexo_cod", "depto_cod", "ano", "semana"]
X = df[columnas].values
Y = df["target"].values
```

El preprocesamiento es idéntico al de la red neuronal: edad a años, sexo y departamento a números, y el target a 0/1. **Diferencia clave:** el árbol **no necesita normalización**. Como toma decisiones por umbrales (por ejemplo «¿año < 2005?»), la escala de cada variable no le afecta.

3. Entrenamiento del árbol (criterio de entropía)

```
miarbol = tree.DecisionTreeClassifier(criterion="entropy", random_state=42)
# miarbol = tree.DecisionTreeClassifier(criterion="entropy", max_depth=5) # (podado)
miarbol = miarbol.fit(X, Y)
print("Accuracy:", 100*miarbol.score(X, Y))
```

criterion="entropy": en cada nodo, el árbol elige la pregunta que más reduce la **entropía** (el «desorden» o mezcla de clases). Es decir, busca la división que deja los grupos lo más «puros» posible (idealmente, AGUDA por un lado y ERUPTIVA por otro). **fit** construye el árbol automáticamente a partir de los datos.

*La segunda línea, comentada, es la versión **podada** (max_depth=5): limitaría la profundidad para evitar el sobreajuste. Igual que en el ejemplo del profesor, se deja como referencia pero NO se usa: el*

árbol crece sin límite.

4. Visualización del árbol

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
plot_tree(miarbol, max_depth=3, feature_names=columnas,
          class_names=["AGUDA", "ERUPTIVA"], filled=True)
plt.show()
```

plot_tree dibuja el árbol. Como el árbol completo es enorme (memoriza el entrenamiento), mostramos solo los **primeros 3 niveles** con **max_depth=3**. Cada caja indica la pregunta, la entropía, cuántos casos llegan y la clase predominante; **filled=True** colorea según la clase.

5. Evaluación con el conjunto de prueba

```
Xtest, Ytest = preparar(testing)
print("Accuracy prueba:", 100*miarbol.score(Xtest, Ytest))

M = confusion_matrix(Ytest, miarbol.predict(Xtest))
sensibilidad = M[0,0] / (M[0,0] + M[1,0])
especificidad = M[1,1] / (M[1,1] + M[0,1])
```

Evaluamos sobre datos que el árbol **no vio**. La **sensibilidad** mide qué tan bien detecta la clase AGUDA; la **especificidad**, qué tan bien detecta ERUPTIVA. Comparar la exactitud en entrenamiento vs. prueba revela si el modelo memorizó o realmente aprendió.

6. Guardado y uso en la web

```
import pickle
pickle.dump(miarbol, open("miarbolcarrion.pkl", "wb")) # modelo entrenado

# Exportación a JSON (nodos del árbol) para recorrerlo en el navegador:
# children_left, children_right, feature, threshold, value -> ad.json
```

El árbol se guarda con **pickle** (formato de Python). Para la web se exporta además a un **JSON** con la estructura de nodos, y un recorrido de ~20 líneas en JavaScript reproduce exactamente las mismas decisiones. Se verificó que coincide al 100% con scikit-learn sobre las 9,224 filas de prueba.

Resultado e interpretación

El árbol obtiene cerca de **93% en entrenamiento pero solo 67% en prueba**. Esa gran diferencia es señal de **sobreajuste (overfitting)**: el árbol **memorizó** el entrenamiento (creando reglas para casos rarísimos) en lugar de aprender patrones generales. Por eso puede dar **100% de «certeza»** en casos atípicos (por ejemplo un caso en Lima, donde la enfermedad casi no existe), aunque esa certeza se base en muy pocos ejemplos.

Conclusión de la comparativa: sobre los mismos datos, el Árbol de Decisión memoriza (alta exactitud en entrenamiento, baja en prueba) mientras que la Red Neuronal generaliza mejor (exactitud estable ~72%). El árbol gana en **interpretabilidad** (sus reglas se pueden leer y dibujar); la red gana en **generalización y prudencia** de sus probabilidades.